

MODULE 7 · MULTI-TENANCY & SECURITY

Certificate Inspection

Reading X.509 certificates and keys with OpenSSL

WHY IT MATTERS

TLS is everywhere in a cluster — you need to read what's inside it

Ingress TLS, the API server's own certificate, kubelet client certs — all of it is **X.509**. `openssl x509 -text` is the one skill that decodes every one of them.

Whether a certificate came from cert-manager, kubeadm, or a CSR you signed by hand, it's the same structure underneath. Learn to read one and you can read them all.

BY THE END YOU CAN

- Explain what a certificate actually contains
- Decode a TLS Secret and read the cert inside it
- Tell Subject from SAN — and know which matters
- Check whether a cert is about to expire

CONCEPT

A certificate is a public key + an identity, signed by someone who vouches for it

X.509 binds a **public key** to an **identity** — a Subject and/or a list of Subject Alternative Names. That binding only means something once a **Certificate Authority (CA)** signs it with its own private key.

Anyone holding the CA's public key can verify the signature, proving the binding hasn't been tampered with. The certificate's matching **private key** never travels — its holder proves identity by signing or decrypting, never by showing the certificate.

Self-signed just means the issuer and the subject are the same identity — no third party vouches for it.



One block, every field you need

```
Issuer:                # self-signed → issuer == subject
Subject:               # blank — no CN set
Validity
  Not Before: Jul  1 00:00:00 2026 GMT
  Not After : Sep 29 00:00:00 2026 GMT
Subject Public Key Info:
  Public Key Algorithm: rsaEncryption
X509v3 Subject Alternative Name:
  DNS:lab.example.com, DNS:*.lab.example.com
Signature Algorithm: sha256WithRSAEncryption
```

- **Issuer** — identity of whoever signed it
- **Subject** — legacy identity field; often blank on self-signed certs
- **Validity** — the Not Before / Not After window
- **SAN** — the DNS names clients actually check
- **Algorithms** — key type and how the signature was made

Produced by `openssl x509 -text -noout` — every other check on this deck shortcuts one of these fields.

A Secret is base64 — decode it straight into openssl

A `kubernetes.io/tls` Secret stores `tls.crt` and `tls.key` as base64 text. `kubectl`'s `jsonpath` pulls the field out; `base64 -d` feeds it straight into `openssl x509` — no temp files needed.

```
$ kubectl get secret lab-cert-tls -n training \
  -o jsonpath='{.data.tls\.crt}' \
  | base64 -d | openssl x509 -text -noout
```

Swap the trailing flags to ask a narrower question of the same decoded cert:

```
$ ... | openssl x509 -enddate -noout
# just the expiry
```

```
$ ... | openssl x509 -subject -issuer -noout
# who / for whom
```

```
$ ... | openssl x509 -noout -ext subjectAltName
# just the SANs
```

Self-signed certs often ship an empty Subject — SAN is what TLS actually checks

FIELD	WHAT IT HOLDS	TRUST IT FOR HOSTNAME MATCHING?
Subject (CN)	Legacy identity field, human-readable	No — deprecated; modern clients ignore it
Subject Alternative Name (SAN)	List of DNS names / IPs the cert is valid for	Yes — this is what a TLS client matches against the hostname
Issuer	Identity of whoever signed the cert	Compare against a trusted CA, not the cert itself

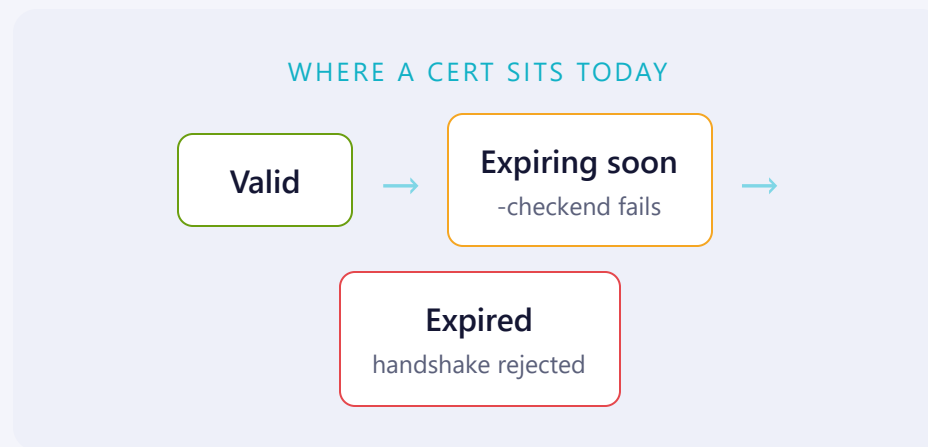
WHY THE SUBJECT CAN BE BLANK

A self-signed cert's issuer and subject are the same identity — with no `commonName` set, `subject=` comes back empty. That's normal, not a defect; identity rides entirely on the SAN list.

Not Before / Not After — and how you'd gate on it

Not After is the field that matters: once past it, TLS clients refuse the connection outright. `-enddate` prints it as a human-readable date; `-checkend` answers the operational question directly.

```
$ ... | openssl x509 -enddate -noout  
notAfter=Sep 29 00:00:00 2026 GMT  
  
$ ... | openssl x509 -checkend 86400 -noout  
Certificate will not expire # exit 0
```



Where people trip

- **Comparing Subject, not SAN.** A blank CN looks broken but is normal — hostname trust rides on the SAN list.
- **Forgetting -noout.** Without it, `openssl x509 -text` also dumps the raw PEM block underneath.
- **-text parses fine on an expired cert.** Parsing isn't validating — check `-enddate` or `-checkend` explicitly.
- **Mismatched key/cert.** A handshake fails silently until you compare the `-pubkey` hashes yourself.

Commands to keep at hand

```
$ openssl x509 -text -noout -in cert.pem           # full human-readable dump
$ openssl x509 -enddate -noout -in cert.pem        # expiry only
$ openssl x509 -subject -issuer -noout -in cert.pem # who / for whom
$ openssl x509 -noout -ext subjectAltName -in cert.pem # SANs only
$ openssl x509 -checkend 86400 -noout -in cert.pem  # exits 1 if expiring <1 day
$ openssl verify -CAfile ca.crt cert.pem           # chain of trust
```

Tip: pair `-pubkey` | `openssl md5` on the cert and the key — matching hashes mean they belong together.

RECAP

Certificate inspection in one breath

- A cert = public key + identity, signed by a CA
- `openssl x509 -text -noout` decodes any of them
- Trust the SAN, not the Subject — self-signed Subjects are often blank
- `-enddate` / `-checkend` tell you if it's still good

HANDS-ON

Now run Lab 7.6
[labs/7.6/](#)

Next: [8.1 — Kustomize](#)